

Fiche réponses — TP Migrer StayBook vers Kubernetes avec Argo CD



À rendre via le dépôt GitHub (REPONSES.md)

Enseignant : Youssef YOUNES

Année universitaire 2026-2027

Polys générés à l'aide d'une IA — contenu relu et validé par l'enseignant.

Consigne : cette fiche reprend les 34 questions du TP. Rédigez vos réponses dans `REPONSES.md` et committez-le dans votre dépôt `staybook-gitops`.

Nom(s) :

Dépôt GitHub :

Étape 1 — Docker Compose : le point de départ

Question 1

Dans ce `docker-compose.yml`, repérez tout ce que Compose fait **pour vous** et que vous n'aurez plus « gratuitement » dans Kubernetes : l'ordre de démarrage, le réseau, les volumes, les ports, les variables d'environnement, la construction des images. Pour chacun, notez le mot-clé Compose correspondant.

Question 2

Pourquoi faut-il impérativement faire `docker compose down` avant de continuer ? (Deux raisons : l'une concerne un port, l'autre la mémoire.)

Étape 2 — Kubernetes + Argo CD : la migration

Question 3

En mode push, qui détient les droits d'administration sur le cluster ? Citez deux risques concrets que le modèle pull réduit.

Question 4

Un collègue modifie « à la main » le nombre de répliques d'un déploiement avec `kubectl scale` sur un cluster géré par Argo CD. D'après le principe 4, que va-t-il se passer ? Est-ce souhaitable ?

Question 5

Quel composant est le seul à parler à GitHub ? Lequel est le seul à créer des ressources dans le cluster ?

Question 6

Le cluster est « vierge ». Quels namespaces existent déjà, et à quoi servent-ils ? Combien de pods tournent, et pourquoi n'y en a-t-il aucun dans `default` ?

Question 7

Pourquoi le script tague les images `v1` plutôt que `latest` ? Cherchez la règle de Kubernetes sur `imagePullPolicy` quand le tag est `latest` : que se passerait-il au démarrage d'un pod ?

Question 8

Où sont physiquement les images maintenant ? Vérifiez avec `docker exec argocd-tp-control-plane crictl images | grep staybook`. Qu'est-ce que `crictl`, et pourquoi n'est-ce pas `docker images` ?

Question 9

Combien d'objets vont être créés ? Parmi les types listés, trois ne sont pas des objets Kubernetes « standard ». Lesquels, et à quoi servent-ils ? (Indice : `CustomResourceDefinition`.)

Question 10

Pourquoi est-il préférable de passer par Kustomize plutôt que de faire `kubectl apply -f install.yaml` puis `kubectl patch` sur le Service ? Pensez au jour où vous mettrez Argo CD à jour.

Question 11

Vérifiez votre tableau de la partie 1 : quel composant est un `StatefulSet` ? Avez-vous les sept composants ?

Question 12

Essayez `kubectl apply -k argocd-install sans --server-side` (c'est sans danger, tout existe déjà). Quelle erreur obtenez-vous, sur quel objet ? Cherchez la cause (`last-applied-configuration`).

Question 13

Pourquoi la commande se termine-t-elle par `| base64 -d` ? Que verriez-vous sans ? Un Secret Kubernetes est-il chiffré ?

Question 14

Que signifient `--plaintext` et `--grpc-web` ? Que se passe-t-il sans `--plaintext` (essayez) ? Ces options seraient-elles acceptables en production ?

Question 15

Regardez `apps/staybook/00-secrets.yaml` : les mots de passe PostgreSQL sont en clair dans Git. Est-ce acceptable ici ? Et en entreprise ? Citez une solution.

Question 16

Notez l'erreur. Pourquoi GitHub répond-il « Repository not found » et non « accès refusé » ?

Question 17

Sous quelle forme Argo CD a-t-il stocké ces identifiants dans le cluster ? Trouvez l'objet avec `kubectl get secrets -n argocd -l argocd.argoproj.io/secret-type=repository` et listez ses clés (`-o jsonpath='{.data}'` puis décodez url et username, pas password).

Question 18

Comparez trois façons de s'authentifier (token personnel, deploy key SSH, GitHub App) : portée, rotation, ce qui se passe quand la personne quitte l'entreprise. Laquelle recommanderiez-vous pour 200 dépôts ?

Question 19

Combien de ressources Argo CD compte-t-il déployer ? Retrouvez chacune dans les fichiers de `apps/staybook/`.

Question 20

Les ressources ne sont pas déployées toutes en même temps. Ouvrez les manifests et trouvez l'annotation qui pilote cet ordre. Quel ordre a été choisi, et pourquoi ? À quoi cela correspondait-il dans `docker-compose.yml` ?

Question 21

« OutOfSync » et « Healthy » sont deux statuts indépendants. Donnez un exemple d'application `Synced` mais `Degraded`, et un exemple `OutOfSync` mais `Healthy`.

Question 22

Vous venez de faire un `kubectl apply ...` n'est-ce pas contraire au GitOps ? Pourquoi ce dernier `apply` est-il inévitable, et comment réduire cette « amorce » à un seul objet pour tout un cluster ? (Cherchez « App of Apps ».)

Question 23

Reprenez votre réponse à la question 1 et complétez la colonne de droite en vous appuyant sur les manifests fournis.

Question 24

Pourquoi les deux Applications pointent-elles vers le **même namespace** `staybook` ? Que se passerait-il si le `concierge` était dans un namespace `concierge` ? (Indice : regardez `RAG_URL` dans le manifest de la gateway.)

Question 25

Ouvrez `apps/concierge/20-rag-service.yaml`. Que font les deux `initContainers` ? En quoi est-ce différent des sync waves de la question 20 ? Pourquoi avoir les deux ?

Question 26

Ouvrez `apps/concierge/11-ollama.yaml`. Où sont stockés les modèles téléchargés ? Supprimez le pod Ollama (`kubectl -n staybook delete pod -l app=ollama`) et vérifiez avec `kubectl -n staybook exec deploy/ollama -- ollama list` que les modèles sont toujours là. Qu'est-ce qui les a conservés ? Que signifie `strategy: Recreate` ?

Question 27

Toujours dans ce fichier, lisez le bloc `resources`. Que se passe-t-il si Ollama dépasse la limite mémoire ? Et si le nœud n'a pas les 2 Gi demandés ?

Étape 3 — GitOps : Git pilote le cluster

Question 28

Sans `--refresh`, au bout de combien de temps Argo CD aurait-il vu le commit ? D'où vient cette valeur (`ConfigMap argocd-cm`, clé `timeout.reconciliation`) ? Quel mécanisme permettrait de réagir en quelques secondes sans interroger GitHub en boucle, et pourquoi ne peut-on pas l'utiliser ici ?

Question 29

Que s'est-il passé, et en combien de temps ? Quel composant a agi ? Retrouvez sa trace : `kubectl -n argocd logs sts/argocd-application-controller | grep staybook | tail`. Essayez plus fort : `kubectl -n staybook delete deploy gateway`. Le site est-il resté accessible ?

Question 30

Décrivez précisément la chaîne d'événements entre votre `git push` et le nouveau titre dans le navigateur. Qui a fait quoi ? Où est passé l'ancien pod ? (`kubectl -n staybook get rs`)

Question 31

Dans une entreprise, qui construirait et chargerait l'image à votre place, et où ? Quelle partie de ce que vous venez de faire relève de la **CI**, et quelle partie du **CD** ?

Question 32

Qu'est-il arrivé au ConfigMap ? Que se serait-il passé avec `prune: false` ? Pourquoi `prune` est-il désactivé par défaut dans Argo CD ?

Question 33

Après le rollback (a), l'auto-sync est-il toujours actif ? Pourquoi Argo CD fait-il cela ? Laquelle des deux approches est « GitOps-compatible » ? Le site est-il resté disponible pendant le mauvais déploiement, et grâce à quoi ?

Question 34

Reliez chaque entrée de `argocd app history staybook` à un commit de `git log --oneline`. Face à un auditeur qui demande « qui a déployé quoi, quand », que répondez-vous, en combien de commandes ?

Preuves

- Sortie de `argocd app history staybook` :

- Sortie de `argocd app list` :
